

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «КубГУ»)

Факультет компьютерных технологий и прикладной математики
Кафедра вычислительных технологий

ЛАБОРАТОРНАЯ РАБОТА №7
Дисциплина: Платформено-независимое программирование

Работу выполнил: _____ Костров А.А.

Направление подготовки: 02.03.02 Фундаментальная информатика и информационные технологии

Преподаватель: _____ Шиян В.И.

1. Постановка задачи

- 10 окошек (касс).
- Первые 5: среднее время обслуживания 7 минут.
- Вторые 5: среднее время обслуживания 10 минут.
- Клиенты приходят в среднем раз в 10 минут (сценарий 1) и раз в 5 минут (сценарий 2).
- Смоделировать 1 час работы.
- Подсчитать:
 - количество обслуженных клиентов
 - среднюю и максимальную длину очереди

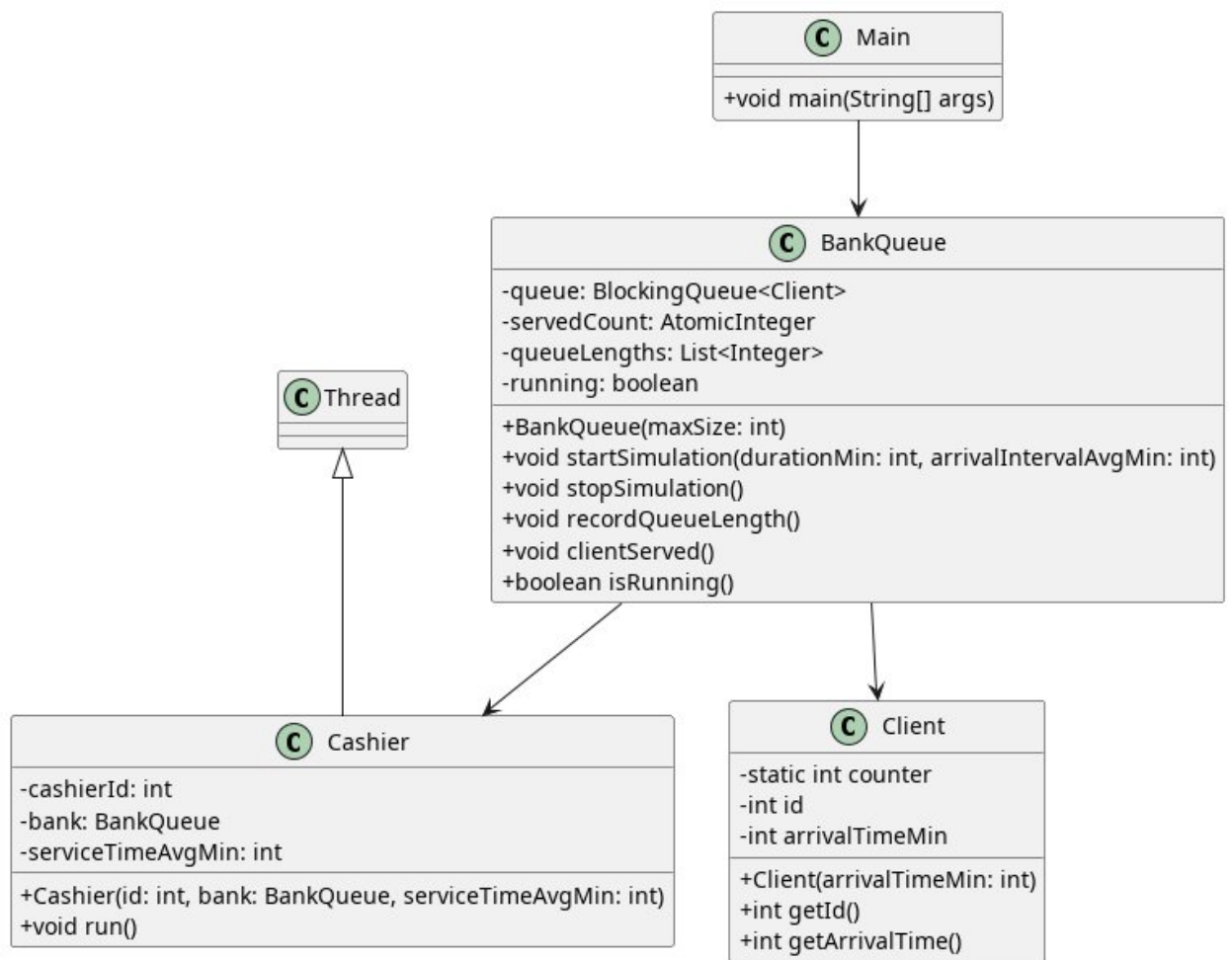
2. Обоснование выбора синхронизатора

Выбран **LinkedBlockingQueue** (реализация **BlockingQueue**).

Причины:

1. **Потокобезопасность** — не требуется дополнительной синхронизации.
2. **Блокирующие операции** — `take()` при пустой очереди автоматически приостанавливает поток-кассира, `put()` — при заполненной (имитация зала ожидания).
3. **Producer-Consumer паттерн** — идеально подходит для задачи: генератор клиентов → очередь → кассы.
4. **Измерение очереди** — `size()` даёт текущую длину для статистики.

3. UML-диаграмма классов



4. Текст программы

Client.java

```
public class Client {
    private static int counter = 0;
    private final int id;
    private final int arrivalTimeMin; // время прихода в минутах от начала

    public Client(int arrivalTimeMin) {
        this.id = ++counter;
        this.arrivalTimeMin = arrivalTimeMin;
    }

    public int getId() {
        return id;
    }

    public int getArrivalTime() {
```

```

        return arrivalTimeMin;
    }

    @Override
    public String toString() {
        return "Client{" + id + ", arrival=" + arrivalTimeMin + "min}";
    }
}

```

BankQueue.java

```

import java.util.ArrayList;
import java.util.List;
import java.util.Random;
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.LinkedBlockingQueue;
import java.util.concurrent.atomic.AtomicInteger;

public class BankQueue {
    private final BlockingQueue<Client> queue;
    private final AtomicInteger servedCount = new AtomicInteger(0);
    private final List<Integer> queueLengths = new ArrayList<>();
    private volatile boolean running = true;

    public BankQueue(int maxQueueSize) {
        this.queue = new LinkedBlockingQueue<>(maxQueueSize);
    }

    public void startSimulation(int durationMinutes, int arrivalIntervalAvgMinutes)
        throws InterruptedException {
        servedCount.set(0);
        queue.clear();
        queueLengths.clear();
        running = true;

        System.out.printf("\n=== СИМУЛЯЦИЯ: %d мин, клиент раз в %d мин\n",
            durationMinutes, arrivalIntervalAvgMinutes);

        // Запуск 10 касс
        Cashier[] cashiers = new Cashier[10];
        for (int i = 0; i < 5; i++) {
            cashiers[i] = new Cashier(i + 1, this, 7);
            cashiers[i].start();
        }
    }
}

```

```

    }
    for (int i = 5; i < 10; i++) {
        cashiers[i] = new Cashier(i + 1, this, 10);
        cashiers[i].start();
    }

    // Генератор клиентов
    Thread generator = new Thread(() -> {
        Random rand = new Random();
        int currentTime = 0;
        int clientCounter = 0;

        while (running && currentTime <= durationMinutes) {
            try {
                clientCounter++;
                Client client = new Client(currentTime);
                queue.put(client); // блокируется, если очередь полна

                synchronized (queueLengths) {
                    queueLengths.add(queue.size());
                }

                System.out.printf("[%d мин] Пришёл %s. Очередь: %d\n",
                    currentTime, client, queue.size());

                // Следующий клиент с вариацией ±50%
                int nextInterval = arrivalIntervalAvgMinutes + rand.nextInt(arrivalIntervalAvgMinutes) - arrivalIntervalAvgMinutes / 2;
                nextInterval = Math.max(1, nextInterval);

                // Масштаб: 1 минута = 10 мс (для наглядности)
                Thread.sleep(nextInterval * 10L);
                currentTime += nextInterval;

            } catch (InterruptedException e) {
                break;
            }
        }
        running = false;
    });
    generator.start();

    // Ждём симуляцию
    Thread.sleep(durationMinutes * 10L);

```

```

// Остановка
running = false;
generator.interrupt();
for (Cashier c : cashiers) {
    c.interrupt();
}
for (Cashier c : cashiers) {
    c.join();
}

// Статистика
System.out.println("\n--- СТАТИСТИКА ---");
System.out.println("Обслужено клиентов: " + servedCount.get());
if (!queueLengths.isEmpty()) {
    double avg = queueLengths.stream().mapToInt(Integer::intValue).average().orElse(0);
    int max = queueLengths.stream().max(Integer::compareTo).orElse(0);
    System.out.printf("Средняя длина очереди: %.2f\n", avg);
    System.out.println("Максимальная длина очереди: " + max);
}

public void serveClient(Cashier cashier) throws InterruptedException {
    Client client = queue.take();
    Random rand = new Random();
    int serviceTime = cashier.getServiceTimeAvg() + rand.nextInt(3) - 1; // -1..+1
    мин
    serviceTime = Math.max(3, serviceTime);

    System.out.printf(" [Касса %d] Обслуживает %s (%d мин)\n",
        cashier.getCashierId(), client, serviceTime);

    Thread.sleep(serviceTime * 10L);

    servedCount.incrementAndGet();
    System.out.printf(" [Касса %d] Закончил. Всего обслужено: %d\n",
        cashier.getCashierId(), servedCount.get());
}

public boolean isRunning() {
    return running;
}
}

```

Cashier.java

```
public class Cashier extends Thread {
    private final int cashierId;
    private final BankQueue bank;
    private final int serviceTimeAvg;

    public Cashier(int cashierId, BankQueue bank, int serviceTimeAvg) {
        this.cashierId = cashierId;
        this.bank = bank;
        this.serviceTimeAvg = serviceTimeAvg;
    }

    public int getCashierId() {
        return cashierId;
    }

    public int getServiceTimeAvg() {
        return serviceTimeAvg;
    }

    @Override
    public void run() {
        while (bank.isRunning() && !Thread.currentThread().isInterrupted()) {
            try {
                bank.serveClient(this);
            } catch (InterruptedException e) {
                break;
            }
        }
    }
}
```

Main.java

```
public class Main {  
    public static void main(String[] args) throws InterruptedException {  
        BankQueue bank = new BankQueue(50);  
        bank.startSimulation(60, 10); // 1 час, клиент раз в 10 мин  
  
        BankQueue bank2 = new BankQueue(50);  
        bank2.startSimulation(60, 5); // 1 час, клиент раз в 5 мин  
    }  
}
```

Результаты работы программы

СИМУЛЯЦИЯ: 60 мин, клиент раз в 10 мин:
[0 мин] Пришёл Client{1, arrival=0min}. Очередь: 1
[Касса 1] Обслуживает Client{1, arrival=0min} (7 мин)
[Касса 1] Закончил. Всего обслужено: 1
[9 мин] Пришёл Client{2, arrival=9min}. Очередь: 1
[Касса 3] Обслуживает Client{2, arrival=9min} (6 мин)
[Касса 3] Закончил. Всего обслужено: 2
[21 мин] Пришёл Client{3, arrival=21min}. Очередь: 1
[Касса 5] Обслуживает Client{3, arrival=21min} (8 мин)
[Касса 5] Закончил. Всего обслужено: 3
[34 мин] Пришёл Client{4, arrival=34min}. Очередь: 1
[Касса 4] Обслуживает Client{4, arrival=34min} (7 мин)
[Касса 4] Закончил. Всего обслужено: 4
[47 мин] Пришёл Client{5, arrival=47min}. Очередь: 1
[Касса 2] Обслуживает Client{5, arrival=47min} (8 мин)
[55 мин] Пришёл Client{6, arrival=55min}. Очередь: 1
[Касса 2] Закончил. Всего обслужено: 5
[Касса 6] Обслуживает Client{6, arrival=55min} (9 мин)

СТАТИСТИКА:
Обслужено клиентов: 5
Средняя длина очереди: 1.00
Максимальная длина очереди: 1

СИМУЛЯЦИЯ: 60 мин, клиент раз в 5 мин:
[0 мин] Пришёл Client{7, arrival=0min}. Очередь: 1
[Касса 1] Обслуживает Client{7, arrival=0min} (7 мин)
[5 мин] Пришёл Client{8, arrival=5min}. Очередь: 1
[Касса 2] Обслуживает Client{8, arrival=5min} (7 мин)
[Касса 1] Закончил. Всего обслужено: 1
[11 мин] Пришёл Client{9, arrival=11min}. Очередь: 1
[Касса 3] Обслуживает Client{9, arrival=11min} (8 мин)
[Касса 2] Закончил. Всего обслужено: 2
[17 мин] Пришёл Client{10, arrival=17min}. Очередь: 1
[Касса 4] Обслуживает Client{10, arrival=17min} (6 мин)

[Касса 3] Закончил. Всего обслужено: 3
[22 мин] Пришёл Client{11, arrival=22min}. Очередь: 1
[Касса 5] Обслуживает Client{11, arrival=22min} (8 мин)
[Касса 4] Закончил. Всего обслужено: 4
[29 мин] Пришёл Client{12, arrival=29min}. Очередь: 1
[Касса 6] Обслуживает Client{12, arrival=29min} (9 мин)
[Касса 5] Закончил. Всего обслужено: 5
[36 мин] Пришёл Client{13, arrival=36min}. Очередь: 1
[Касса 7] Обслуживает Client{13, arrival=36min} (9 мин)
[Касса 6] Закончил. Всего обслужено: 6
[43 мин] Пришёл Client{14, arrival=43min}. Очередь: 1
[Касса 8] Обслуживает Client{14, arrival=43min} (9 мин)
[Касса 7] Закончил. Всего обслужено: 7
[46 мин] Пришёл Client{15, arrival=46min}. Очередь: 1
[Касса 9] Обслуживает Client{15, arrival=46min} (11 мин)
[52 мин] Пришёл Client{16, arrival=52min}. Очередь: 1
[Касса 10] Обслуживает Client{16, arrival=52min} (10 мин)
[Касса 8] Закончил. Всего обслужено: 8
[Касса 9] Закончил. Всего обслужено: 9

СТАТИСТИКА:

Обслужено клиентов: 9

Средняя длина очереди: 1.00

Максимальная длина очереди: 1